

Konspekt projektu badawczego

Zastosowanie Monte Carlo Tree Search (MCTS/UCT) do gry w warcaby 10×10 bez obowiązkowego bicia z damkami bijącymi po całej diagonalu

Jakub Kindracki

Wiktor Kobielski

5 maja 2026

1 Opis problemu

Przedmiotem projektu jest zastosowanie algorytmu Monte Carlo Tree Search (MCTS), w szczególności jego wariantu Upper Confidence Bound applied to Trees (UCT), do stworzenia sztucznej inteligencji grającej w wybraną grę dwuosobową z pełną informacją. Grą wybraną do badań są warcaby na planszy 10×10 z modyfikacjami klasycznych zasad: *bicie nie jest obowiązkowe* oraz *damki (latające) biją wzdłuż całej diagonalu*, co odpowiada zasadom zbliżonym do warcabów międzynarodowych w aspekcie ruchu damki, lecz różnym w aspekcie obowiązku bicia.

Brak obowiązkowego bicia istotnie zwiększa współczynnik rozgałęzienia drzewa gry — w każdej pozycji, w której istnieje bicie, gracz może je wykonać lub wybrać dowolny inny ruch. Oznacza to, że MCTS musi rozważać znacznie szerszy zbiór akcji niż w klasycznych warcabach, a heurystyki ukierunkowane na bicia (typowe dla silników warcabowych) mają ograniczone zastosowanie. Latające damki natomiast wzmacniają wartość promocji i wprowadzają silne, dalekosiężne interakcje pomiędzy figurami, co może wpływać na jakość losowych symulacji. Ta kombinacja zasad czyni grę nietrywialnym polygonem do badania algorytmu MCTS i jego rozszerzeń.

Stan gry reprezentujemy jako planszę 10×10 z 20 pionkami po każdej stronie, ustawionymi na ciemnych polach pierwszych czterech rzędów każdego z graczy. Gra kończy się zwycięstwem strony, której przeciwnik nie ma legalnego ruchu (brak figur lub całkowite zablokowanie). W celu zapewnienia, aby eksperymenty kończyły się w rozsądnym czasie, wprowadzimy limit liczby pól-ruchów bez bicia i bez ruchu pionka — przekroczenie limitu skutkuje remisem (zasada zbliżona do reguły 50 ruchów w szachach).

2 Wstępny przegląd literatury

Algorytm MCTS w jego współczesnej postaci został sformalizowany przez Kocsisa i Szepesváriego [?] jako UCT — zastosowanie strategii UCB1 [?] do równoważenia eksploracji i eksploatacji w drzewie gry. Browne i in. [?] przedstawili obszerny przegląd wariantów MCTS i obszarów ich zastosowań w grach.

Kluczowym osiągnięciem MCTS jest wynik w grze Go: AlphaGo i AlphaZero [?, ?] pokazały, że MCTS w połączeniu z głębokimi sieciami neuronowymi prowadzi do nadludzkiego poziomu gry. W naszym projekcie skupiamy się na "klasycznym" MCTS bez sieci neuronowych, ze względu na zakres przedmiotu i ograniczenia sprzętowe.

W ramach usprawnień UCT zbadamy dwa podejścia z literatury:

- **RAVE / AMAF** (Rapid Action Value Estimation, All Moves As First) – Gelly i Silver [?, ?] zaproponowali heurystykę współdzielenia statystyk ruchów, znacząco przyspieszającą zbieżność wartości akcji w rzadko odwiedzanych węzłach.
- **Progressive Bias / Progressive Widening** – Chaslot i in. [?] wprowadzili wzmocnienie reguły wyboru węzła członem heurystycznym, którego wpływ maleje wraz z liczbą wizyt.

W kontekście warcabów referencyjnym osiągnięciem jest program CHINOOK [?], który dowiódł, że warcaby angielskie (8×8 , z obowiązkowym biciem) są *rozwiązane* (gra perfekcyjna kończy się remisem). Warcaby 10×10 pozostają znacznie bardziej złożone obliczeniowo i nie zostały rozwiązane. Programy oparte o $\alpha\beta$ -przeszukiwanie z heurystykami (np. KINGSROW, SCAN) dominują w warcabach międzynarodowych, ale MCTS w tym wariancie gry pozostaje stosunkowo słabo zbadany w literaturze, co dodatkowo motywuje nasze badania.

Heurystyczna funkcja oceny dla warcabów – materiał, mobilność, kontrola centrum, struktura tylnego rzędu – jest dobrze opisana m.in. w pracach Samuela [?] i pracach autorów Chinook. Wykorzystamy ją zarówno w gracz czysto heurystycznym (przeciwnik bazowy), jak i w członie progressive bias.

3 Hipotezy badawcze

Zgodnie z wymaganiami projektu stawiamy następujące hipotezy:

H1 (UCT vs heurystyka).

UCT z wystarczającą liczbą iteracji ($\geq 10\,000$ na ruch) gra istotnie skuteczniej od gracza czysto heurystycznego (greedy, jednopoziomowy), osiągając $> 60\%$ wygranych w meczu z naprzemienną zmianą kolorów.

H2 (RAVE).

UCT z rozszerzeniem RAVE wygrywa istotnie częściej z czystym UCT przy tym samym budżecie iteracji, ze względu na lepsze wykorzystanie informacji z symulacji w grze o wysokim współczynniku rozgałęzienia.

H3 (Progressive Bias).

UCT z heurystycznym progressive bias wygrywa istotnie częściej z czystym UCT, a jego przewaga jest największa przy małych budżetach iteracji (gdy losowe symulacje nie wystarczają do oceny pozycji).

H4 (hipoteza autorska – wpływ braku obowiązku bicia).

Brak obowiązkowego bicia istotnie pogarsza skuteczność czystego UCT względem wariantu z obowiązkowym biciem przy tym samym budżecie iteracji. Innymi słowy: większy współczynnik rozgałęzienia wynikający z dobrowolności bicia obniża jakość decyzji UCT bardziej niż wariantów z heurystycznym wsparciem (RAVE, progressive bias). W szczególności oczekujemy, że *względna* przewaga RAVE i progressive bias nad czystym UCT będzie większa w wariancie bez obowiązkowego bicia niż w wariancie klasycznym.

4 Sposób weryfikacji hipotez

4.1 Plan eksperymentów

Każdą hipotezę zweryfikujemy poprzez zorganizowanie turniejów *round-robin* pomiędzy graczami przy ustalonym budżecie iteracji / czasu. Dla każdej pary graczy rozegramy $N \geq 50$ partii (po $N/2$ z każdym kolorem), aby wyeliminować wpływ pierwszego ruchu.

Zbiory danych. Ponieważ jest to gra dwuosobowa, "danymi" są wyniki rozegranych partii. Dla zapewnienia powtarzalności:

- Każdy gracz przyjmuje parametr `seed`; ziarna pochodne wyprowadzamy deterministycznie z ziarna bazowego.
- Każda konfiguracja eksperymentu (para graczy, budżet, wariant zasad) zostanie powtórzona z ≥ 5 różnymi ziarnami bazowymi, co pozwoli oszacować rozrzut wyników.

- Wszystkie partie zostaną zapisane (sekwencja ruchów + ziarno), umożliwiając powtórzenie konkretnego przebiegu.

Mierzone wielkości. Procent wygranych, remisów i przegranych z 95%-przedziałem ufności (Wilson lub bootstrap), średnia długość partii, średni czas na ruch, średnia liczba odwiedzonych węzłów drzewa.

Testy istotności. Dla porównań dwóch graczy zastosujemy dwustronny test proporcji (lub bootstrap przedziału ufności na różnicy procentów wygranych). Hipotezę uznajemy za potwierdzoną, jeśli $p < 0,05$ i efekt jest spójny dla różnych ziaren.

Plan opracowania wyników. Wyniki przedstawimy w postaci: wykresów słupkowych z przedziałami ufności (porównania graczy), wykresów krzywych (skuteczność vs budżet iteracji), tabel zbiorczych w raporcie. Surowe dane zostaną dołączone jako pliki JSON/CSV.

4.2 Testy z udziałem ludzi

Zaimplementujemy interfejs przeglądarkowy (Flask + HTML/JS) umożliwiający grę człowiek-komputer. Przeprowadzimy testy z co najmniej 5 osobami (różny poziom znajomości warcabów). Każda osoba rozegra po jednej partii z każdym wariantem AI; zbierzemy ankietę dotyczącą subiektywnej oceny "ludzkiego" charakteru i siły gry, a także wyniki obiektywne.

5 Harmonogram działań

Termin	Zadanie
Tydzień 1–2	Implementacja silnika gry (warcaby 10×10 z wybranymi zasadami), testy jednostkowe generatora ruchów.
Tydzień 3	Implementacja czystego UCT i interfejsu gracza, gracza losowego i heurystycznego.
Tydzień 4	Implementacja wariantów UCT: RAVE i progressive bias.
Tydzień 5	Implementacja interfejsu przeglądarkowego, runner eksperymentalny CLI z obsługą ziarna i raportowania.
Tydzień 6	Eksperymenty H1–H3 (porównania graczy AI). Optymalizacja wydajności (parallel root MCTS).
Tydzień 7	Eksperymenty H4 (porównanie wariantów zasad), testy z udziałem 5+ osób.
Tydzień 8	Opracowanie wyników, pisanie raportu, przygotowanie prezentacji.

6 Ogólny projekt techniczny

Język i biblioteki. Implementacja w **Python 3**. Biblioteki:

- **numpy** – reprezentacja planszy, agregacja wyników.
- **multiprocessing** – root parallelization MCTS (niezależne drzewa w procesach roboczych, agregacja liczników wizyt).
- **Flask + HTML/JavaScript** – interfejs przeglądarkowy (uruchamiany z poziomu `python app.py`, cross-platform).
- **matplotlib / pandas** – analiza i wykresy w raporcie.
- **pytest** – testy generatora ruchów.

Architektura kodu.

- `game.py` – silnik gry: stan, generator ruchów (pionki oraz latające damki), bicia łańcuchowe, promocja, detekcja zakończenia.
- `mcts.py` – MCTS z trzema wariantami selekcji (`uct`, `rave`, `progressive`) i równoległością na poziomie pierwiastka.
- `players.py` – abstrakcja gracza: `RandomPlayer`, `HeuristicPlayer`, `MCTSPlayer`.
- `app.py` – serwer Flask, REST API dla UI.
- `experiments.py` – runner CLI eksperymentów (round-robin, kontrola ziarna, eksport JSON/CSV).

Reprodukowalność. Wszyscy gracze przyjmują `seed`. Eksperymenty alternują kolory i używają ziaren wyprowadzonych deterministycznie z bazowego. Pełna konfiguracja partii (gracze, ziarno, zasady) zapisywana jest w wynikach, co pozwala odtworzyć dowolny przebieg.

Wykorzystanie LLM. Duże modele językowe (Claude Code) były i będą wykorzystywane jako asystent programistyczny – generowanie szkielethu kodu, refaktoryzacja, sugerowanie testów. Każde użycie zostanie udokumentowane i podsumowane w raporcie końcowym, zgodnie z wymaganiami projektu. Ostateczna treść raportu, hipotezy, projekt eksperymentów i analiza wyników są dziełem autorów.

Literatura

- [1] L. Kocsis and C. Szepesvári, *Bandit based Monte-Carlo Planning*, ECML, 2006.
- [2] P. Auer, N. Cesa-Bianchi, P. Fischer, *Finite-time Analysis of the Multiarmed Bandit Problem*, Machine Learning, 47(2–3):235–256, 2002.
- [3] C. Browne et al., *A Survey of Monte Carlo Tree Search Methods*, IEEE Trans. on Computational Intelligence and AI in Games, 4(1):1–43, 2012.
- [4] S. Gelly and D. Silver, *Combining online and offline knowledge in UCT*, ICML, 2007.
- [5] S. Gelly and D. Silver, *Monte-Carlo tree search and rapid action value estimation in computer Go*, Artificial Intelligence, 175(11):1856–1875, 2011.
- [6] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, B. Bouzy, *Progressive strategies for Monte-Carlo tree search*, New Mathematics and Natural Computation, 4(3):343–357, 2008.
- [7] D. Silver et al., *Mastering the game of Go with deep neural networks and tree search*, Nature, 529(7587):484–489, 2016.
- [8] D. Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, arXiv:1712.01815, 2017.
- [9] J. Schaeffer, N. Burch, Y. Björnsson, A. Kishimoto, M. Müller, R. Lake, P. Lu, S. Sutphen, *Checkers is Solved*, Science, 317(5844):1518–1522, 2007.
- [10] A. L. Samuel, *Some Studies in Machine Learning Using the Game of Checkers*, IBM Journal of Research and Development, 3(3):210–229, 1959.